

Google Calendar AI Agent

Project Documentation

LangGraph · FastAPI · Google Calendar API · React · TypeScript

4
phases

5
calendar tools

5
eval types

3
stack layers

Contents

1. Project overview
2. Architecture — how it all fits together
3. Phase 1 — Environment & Google Auth
4. Phase 2 — Calendar tools
5. Phase 3 — LangGraph agent
6. Phase 4 — FastAPI backend & React UI
7. Observability with Phoenix
8. Evals & testing
9. File structure & reference
10. Running the project

1. Project overview

This project is a **conversational AI agent** that manages a Google Calendar through natural language. A user can type messages like "Book a meeting tomorrow at 2pm" or "What's on my calendar this week?" and the agent will understand the intent, call the appropriate Google Calendar APIs, and respond naturally.

Why we built this

- To learn how LangGraph agents work — the ReAct (Reason + Act) loop, tool use, and memory
- To understand how to wrap real APIs (Google Calendar) as LLM-callable tools
- To learn how to stream agent output to a modern chat UI in real time
- To build production habits: observability, evals, and structured testing from day one

What the system can do

User says	Agent does	Tools called
"What's on my calendar?"	Lists upcoming events	list_events
"Am I free tomorrow at 3pm?"	Checks availability	check_availability
"Book a meeting at 2pm"	Checks free, then creates	check_availability → create_event
"Move my standup to 11am"	Finds event, updates time	list_events → update_event
"Cancel the standup"	Confirms, then deletes	list_events → delete_event

Technology choices — the why

Technology	Why chosen
LangGraph	Stateful agent graphs with memory. Better than simple LangChain chains because it handles multi-step reasoning loops and persists conversation history.
Claude (Anthropic)	Strong tool-use capabilities and good at parsing natural language time expressions like 'next Friday at noon'.
Google Calendar API	Direct OAuth integration — no third-party middleware. Full CRUD access to real calendar data.
FastAPI	Async Python web framework. Native support for SSE streaming, which is essential for showing the agent's reasoning in real time.
React + TypeScript	Component-based UI with type safety. Vite for fast dev server, Tailwind for styling.
Phoenix (Arize)	Open-source observability. Zero-config LangChain instrumentation — traces every agent run with no vendor lock-in.

2. Architecture — how it all fits together

The system has three layers that work together: a React frontend, a FastAPI backend that runs the LangGraph agent, and the Google Calendar API. The key design decision is using Server-Sent Events (SSE) to stream the agent's reasoning steps to the UI in real time.

Request lifecycle — what happens when you send a message

1. User types a message

The React UI captures the input and calls `sendMessage()` from the `useChat` hook. The user's message is added to the chat immediately (before the API responds) for a responsive feel.

2. POST /chat to FastAPI

The frontend sends the message and `thread_id` to the FastAPI backend. The current datetime is injected by the backend so the agent is always time-aware without the UI needing to handle this.

3. LangGraph agent starts the ReAct loop

The agent receives the message and enters its reasoning loop. The LLM reads the system prompt, the conversation history, and the message to decide what to do next.

4. Tool calls stream back via SSE

As the agent calls tools, the backend emits SSE events for each `tool_call` and `tool_result`. The UI renders these as live activity indicators — the user sees the agent working in real time.

5. Final reply streams back

When the agent has a final answer, a message event is emitted. The UI adds the assistant reply to the chat and closes the stream.

The ReAct loop — the agent's core reasoning pattern

ReAct stands for Reason + Act. The LLM alternates between reasoning about what to do and acting by calling tools. This loop runs until the LLM decides it has enough information to give a final answer.

```
User: 'Book a meeting tomorrow at 2pm'
```

```
Loop iteration 1:
```

```
  LLM thinks: I should check availability first
```

```
  LLM acts:   calls check_availability(start='2026-05-17T14:00:00', end='...')
```

```
  Tool returns: 'You are free between 2pm and 3pm'
```

```
Loop iteration 2:
```

```
  LLM thinks: The slot is free, I can create the event
```

```
  LLM acts:   calls create_event(title='Meeting', start='2026-05-17T14:00:00', ...)
```

```
  Tool returns: 'Event created: Meeting (ID: evt_xyz)'
```

```
Loop iteration 3:
```

```
LLM thinks: I have the result, I can answer the user
LLM acts:   no tool call -> final answer
Response:   'Done! I have booked a meeting for tomorrow at 2pm.'
```

Memory and conversation threads

LangGraph's MemorySaver keeps the full conversation history per thread_id. Each chat session has a unique thread_id, so the agent remembers context across turns within a session. Asking 'cancel it' after creating a meeting works because the agent has the full conversation in its state.

3. Phase 1 — Environment & Google auth

Phase 1 sets up the foundation: a Python virtual environment, Google Cloud project, OAuth 2.0 credentials, and a verified connection to the Google Calendar API.

Google Cloud setup

- Created a Google Cloud project named 'cal-agent'
- Enabled the Google Calendar API (APIs & Services → Library)
- Configured OAuth consent screen (External, scope: .../auth/calendar)
- Created OAuth 2.0 credentials (Desktop app type for local development)
- Added own email as a test user to bypass Google verification
- Downloaded credentials.json — the app's identity with Google

How OAuth 2.0 works in this project

OAuth 2.0 lets the app act on behalf of a Google account without storing the user's password. The flow runs once:

1. Run auth.py
2. Browser opens -> user logs into Google -> grants calendar permission
3. Google returns an authorization code
4. auth.py exchanges the code for access_token + refresh_token
5. Tokens saved to token.json
6. Future runs use token.json directly (auto-refreshes when expired)

Key files

auth.py

Runs the OAuth flow, saves token.json. Run once manually.

credentials.json

Downloaded from Google Cloud. The app's OAuth client identity.

token.json

Auto-generated after auth.py. Contains the access + refresh tokens.

`.env`

Stores ANTHROPIC_API_KEY and file paths. Never committed to git.

■ `credentials.json` and `token.json` must be in `.gitignore`. They are equivalent to passwords for your Google account.

4. Phase 2 — Calendar tools

Calendar tools are Python functions decorated with `@tool` that the LangGraph agent can call. The `@tool` decorator tells LangChain the function's name, description, and argument schema — the LLM reads these to decide when and how to use each tool.

Why docstrings matter

The docstring of each tool is what the LLM reads to understand what the tool does and when to use it. A vague docstring leads to the wrong tool being called. A specific one leads to correct, well-argued tool selection.

```
# Bad docstring - LLM doesn't know when to use this
@tool
def create_event(title, start_time, end_time):
    """Create an event."""

# Good docstring - LLM knows exactly when and how to call this
@tool
def create_event(title: str, start_time: str, end_time: str, ...):
    """
    Create a new calendar event. Use this when the user wants to book,
    schedule, or add a meeting or appointment.

    Args:
        title: Title/summary of the event
        start_time: Start time in ISO 8601 format, e.g. '2026-06-01T10:00:00'
    """
```

The five tools

Tool	Purpose	Key args	Google API call
<code>list_events</code>	Fetch upcoming events	<code>days_ahead</code> , <code>max_results</code>	<code>events().list()</code>
<code>create_event</code>	Book a new meeting	<code>title</code> , <code>start_time</code> , <code>end_time</code>	<code>events().insert()</code>
<code>update_event</code>	Reschedule/rename event	<code>event_id</code> , <code>start_time</code> , <code>title</code>	<code>events().update()</code>

Tool	Purpose	Key args	Google API call
delete_event	Cancel an event	event_id	events().delete()
check_availability	Check if a slot is free	start_time, end_time	freebusy().query()

The _to_rfc3339 helper — timezone handling

The Google Calendar API requires RFC 3339 datetime strings with timezone info. The LLM often passes bare strings like '2026-05-17T14:00:00' with no timezone. The _to_rfc3339 helper ensures timezone info is always attached before any API call:

```
def _to_rfc3339(dt_str: str, tz: ZoneInfo) -> str:
    if dt_str.endswith('Z'):
        return datetime.fromisoformat(dt_str.replace('Z', '+00:00')).isoformat()
    if '+' in dt_str[10:] or (len(dt_str) > 19 and dt_str[19] == '-'):
        return datetime.fromisoformat(dt_str).isoformat()
    # No timezone -> attach user's local timezone
    return datetime.fromisoformat(dt_str).replace(tzinfo=tz).isoformat()
```

5. Phase 3 — LangGraph agent

The LangGraph agent is the brain of the system. It uses a StateGraph to define a loop: the LLM node decides what to do, the tools node executes tool calls, and the loop continues until the LLM produces a final answer.

Key LangGraph concepts

Concept	What it is	In this project
StateGraph	A graph of nodes and edges that defines the agent's flow	Two nodes: llm and tools
MessagesState	Built-in state type that manages a list of messages	Holds the full conversation history
ToolNode	Pre-built node that executes tool calls from the LLM	Runs the 5 calendar tools
tools_condition	Router that checks if the LLM made tool calls	Routes to tools or END
MemorySaver	Checkpointing that persists state per thread_id	Keeps conversation history across turns

The graph definition

```
builder = StateGraph(MessagesState)

# Two nodes
builder.add_node('llm', call_llm) # LLM decides what to do
```

```

builder.add_node('tools', ToolNode(TOOLS)) # executes tool calls

# Always start at LLM
builder.add_edge(START, 'llm')

# After LLM: tool call -> tools node, no tool call -> END
builder.add_conditional_edges('llm', tools_condition)

# After tools: always back to LLM
builder.add_edge('tools', 'llm')

graph = builder.compile(checkpointer=MemorySaver())

```

The system prompt

The system prompt shapes the agent's behaviour. Key rules defined in it:

- Always call `check_availability` before `create_event` to avoid double-booking
- Ask for confirmation before deleting or updating events
- Default to 1-hour duration when user doesn't specify
- Use friendly, concise language in responses
- Current datetime is injected per message so the agent is always time-aware

Conditional routing — how the agent decides what to do next

`tools_condition` is a function that inspects the last message in state. If the LLM's response contains `tool_calls`, it routes to the tools node. If not, it routes to END (final answer). This is the mechanism that creates the ReAct loop.

```

# What tools_condition does internally:
def tools_condition(state: MessagesState):
    last_msg = state['messages'][-1]
    if last_msg.tool_calls: # LLM wants to call a tool
        return 'tools'
    else:                   # LLM has a final answer
        return END

```

6. Phase 4 — FastAPI backend & React UI

Phase 4 wraps the agent in a web API and builds a chat UI. The key technology is Server-Sent Events (SSE) — a one-way HTTP stream that pushes agent updates to the browser in real time as the agent works.

Why SSE instead of WebSockets?

	SSE	WebSockets
Direction	One-way (server → client)	Bi-directional

	SSE	WebSockets
Protocol	Regular HTTP	Upgraded HTTP connection
Complexity	Simple — just a streaming response	More setup required
Use case	Server pushes updates (perfect here)	Real-time two-way chat
Browser support	Native EventSource API	Native WebSocket API

SSE event types

The backend emits four event types during an agent run:

```
event: tool_call      # agent is calling a tool
data: {"tool": "check_availability", "args": {"start_time": "..."}}

event: tool_result    # tool returned a result
data: {"tool": "check_availability", "content": "You're free"}

event: message        # agent has a final reply
data: {"role": "assistant", "content": "You're free at 2pm!"}

event: done           # stream is closing
data: {}
```

Frontend architecture

The React app has a clear separation between smart and display components:

Component / Hook	Type	Responsibility
useChat (hook)	Smart	All state: messages, isLoading, activeTools. Owns the SSE stream logic and fetch call.
ChatWindow	Smart	Calls useChat, owns input state, handles auto-scroll, passes data to display components.
MessageBubble	Display	Renders a single message bubble. Shows tool call indicators and EventCards.
EventCard	Display	Conditionally renders a green card when a tool result looks like a calendar event.

State management in useChat

Three pieces of state power the UI:

messages: `Message[]`

Full conversation history. Each message has id, role, content, and optional toolCalls array attached after the run completes.

isLoading: boolean

True while the SSE stream is open. Controls the typing indicator, disables the input, and shows live tool activity.

activeTools: ToolCall[]

Live tool calls currently in progress. Updates on each SSE event so the user sees check_availability appear, complete with ✓, then create_event appear — in real time.

■ useRef for accumulation during the SSE stream, useState for display. Tool calls are buffered in a ref (no re-renders) and batch-copied to state when the final message arrives.

Vite proxy

The Vite config proxies /chat requests to the FastAPI backend. This means the frontend calls /chat (not http://localhost:8000/chat), avoiding CORS issues during development and making deployment easier — only one URL to update.

```
// vite.config.ts
server: {
  proxy: {
    '/chat': 'http://localhost:8000'
  }
}
```

7. Observability with Phoenix

Phoenix (Arize) provides trace-level visibility into every agent run. It auto-instruments LangChain/LangGraph via OpenTelemetry — no manual span creation needed. Every LLM call, tool invocation, and agent decision appears in the Phoenix UI at http://localhost:6006.

What a trace shows

- The full conversation input — what the user asked
- Every LLM call — model, token count, latency, finish reason
- Every tool call — name, args, result, latency
- The complete message chain — each step in the ReAct loop
- End-to-end latency for the full agent run

Setup

```
# Must be BEFORE any LangChain imports
import phoenix as px
from phoenix.otel import register
from openinference.instrumentation.langchain import LangChainInstrumentor

px.launch_app() # starts UI at http://localhost:6006
```

```

tracer_provider = register(
    project_name='cal-agent',
    endpoint='http://localhost:6006/v1/traces',
    protocol='http/protobuf',
)

LangChainInstrumentor().instrument(tracer_provider=tracer_provider)

# Now import agent code
from agent.graph import graph

```

■ The Phoenix setup must come before any LangChain imports. If LangChain is imported first, the instrumentation patches nothing and no traces appear.

Why observability matters

Without traces, debugging a misbehaving agent is guesswork. Traces answer: did the agent call the right tool? Did it pass the correct args? How many LLM hops did it take? Where did the latency come from? For production multi-agent systems these questions are critical.

8. Evals & testing

Evals are automated tests that measure how well the agent performs. Unlike traditional unit tests (binary pass/fail), evals grade agent behaviour — tool selection, tool arguments, response quality, and the sequence of steps taken.

The five eval types used

Type	Checks	Example
Exact match	Correct tool selected	'Book meeting' → create_event called
Trajectory	Tools in right order	check_availability before create_event
Arg validation	Correct args passed	start_time = correct ISO datetime
Contains	Key info in reply	'conflict' or 'busy' in reply when slot taken
LLM-as-judge	Subjective quality	Was conflict clearly explained? (scored 1-5)

The mock fixture — why it matters

All evals use a pytest fixture that mocks the Google Calendar API. This means evals never touch real calendar data, run in milliseconds, and can be run safely in CI without any Google credentials.

```

@pytest.fixture(autouse=True)
def mock_calendar(mock):
    mocker.patch('tools.calendar_tools.get_service',
        return_value=MagicMock(...))

```

```
# Every test in evals/ automatically gets the mock
# No real Google API calls ever happen during testing
```

The five evals written

`test_booking_selects_create_event`

Verifies the agent calls `create_event` when asked to book a meeting.

`test_checks_availability_before_booking`

Verifies `check_availability` is called before `create_event` — every time.

`test_no_booking_when_busy`

Verifies the agent does not book when the slot is already taken, and mentions the conflict in its reply.

`test_delete_asks_confirmation`

Verifies the agent asks 'are you sure?' before calling `delete_event`.

`test_list_mentions_event_title`

Verifies the agent's reply includes the event title when listing events.

✓ Run evals after every change to the system prompt or tools. A score drop tells you something broke before it reaches a user.

9. File structure & reference

Complete project structure

```
cal-agent/
├── .env                # secrets (never commit)
├── credentials.json    # Google OAuth client (never commit)
├── token.json          # OAuth tokens (never commit)
├── requirements.txt     # Python dependencies
├── conftest.py         # pytest path setup
├──
├── auth.py            # OAuth flow - run once
├── test_calendar.py    # verify Calendar API works
├── chat.py            # CLI to test agent in terminal
├──
├── agent/
│   ├── __init__.py
│   └── graph.py        # LangGraph StateGraph definition
├──
├── tools/
│   └── __init__.py
```

```

■   ■■■ calendar_tools.py      # 5 @tool functions + helpers
|
■■■ backend/
■   ■■■ main.py                # FastAPI app + SSE streaming endpoint
|
■■■ evals/
■   ■■■ __init__.py
■   ■■■ helpers.py            # run_agent() + llm_judge()
■   ■■■ test_agent.py         # 5 pytest eval cases
|
■■■ frontend/
    ■■■ vite.config.ts        # Vite + Tailwind + proxy config
    ■■■ package.json
    ■■■ src/
        ■■■ App.tsx
        ■■■ types.ts          # Message, ToolCall type definitions
        ■■■ hooks/
            ■   ■■■ useChat.ts  # SSE stream + all chat state
        ■■■ components/
            ■■■ ChatWindow.tsx
            ■■■ Message.tsx
            ■■■ EventCard.tsx

```

Python dependencies

Package	Version	Purpose
langgraph	>=0.2.0	Agent graph, state management, memory
langchain	>=0.3.0	Base LangChain framework
langchain-core	>=0.3.0	Tools, messages, runnables
langchain-anthropic	>=0.2.0	Claude model integration
langchain-community	>=0.3.0	Community integrations
google-auth-oauthlib	>=1.2.0	Google OAuth 2.0 flow
google-auth-httpplib2	>=0.2.0	HTTP transport for Google auth
google-api-python-client	>=2.140.0	Google Calendar API client
fastapi	>=0.115.0	Web framework + SSE streaming
uvicorn	>=0.30.0	ASGI server to run FastAPI
python-dotenv	>=1.0.0	Load .env file into environment
arize-phoenix	>=15.0.0	Observability UI + trace collection

Package	Version	Purpose
openinference-instrumentation-langchain	>=0.1.0	Auto-instrument LangChain
opentelemetry-sdk	>=1.20.0	OpenTelemetry tracing SDK
opentelemetry-exporter-otlp-proto-http	>=1.20.0	OTLP HTTP exporter
pytest	>=8.0.0	Test runner for evals
pytest-mock	>=3.0.0	Mock fixtures for testing
pytest-json-report	>=1.5.0	JSON output for CI scoring

10. Running the project

First-time setup

```
# 1. Clone / create project folder
mkdir cal-agent && cd cal-agent

# 2. Create and activate virtual environment
python3.13 -m venv .env
source .env/bin/activate

# 3. Install Python dependencies
pip install -r requirements.txt

# 4. Create .env file
echo 'ANTHROPIC_API_KEY=sk-ant-...' > .env
echo 'GOOGLE_CREDENTIALS_PATH=./credentials.json' >> .env
echo 'GOOGLE_TOKEN_PATH=./token.json' >> .env

# 5. Download credentials.json from Google Cloud Console
#   (APIs & Services -> Credentials -> Download)

# 6. Run OAuth flow (opens browser, saves token.json)
python auth.py

# 7. Verify Google Calendar connection
python test_calendar.py

# 8. Set up frontend
cd frontend && npm install && cd ..
```

Running the full stack

```
# Terminal 1 - FastAPI backend (port 8000)
uvicorn backend.main:app --reload --port 8000
```

```
# Terminal 2 - React frontend (port 5173)
cd frontend && npm run dev

# Terminal 3 - CLI testing (optional)
python chat.py

# Open the app
open http://localhost:5173

# Open Phoenix traces
open http://localhost:6006
```

Running evals

```
# Run all evals
pytest evals/test_agent.py -v

# Run a specific eval
pytest evals/test_agent.py::test_booking_selects_create_event -v

# Run with output (useful for debugging)
pytest evals/test_agent.py -v -s

# Stop on first failure
pytest evals/test_agent.py -x
```

Environment variables reference

Variable	Required	Description
ANTHROPIC_API_KEY	Yes	Claude API key from console.anthropic.com
GOOGLE_CREDENTIALS_PATH	Yes	Path to credentials.json (default: ./credentials.json)
GOOGLE_TOKEN_PATH	Yes	Path to token.json (default: ./token.json)
LANGFUSE_PUBLIC_KEY	No	Langfuse tracing (optional, for Langfuse users)
LANGFUSE_SECRET_KEY	No	Langfuse tracing (optional)
LANGFUSE_HOST	No	Langfuse host URL (optional)

Common errors and fixes

Error	Cause	Fix
HttpError 403 Calendar API disabled	Google Calendar API not enabled in Cloud Console	APIs & Services -> Library -> Enable Google Calendar API

Error	Cause	Fix
HttpError 400 Bad Request (freebusy)	Datetime string missing timezone info	_to_rfc3339 helper now handles this — ensure it is called
ModuleNotFoundError: No module named 'agent'	pytest can't find project root	Add conftest.py with sys.path.insert(0, project_root)
redirect_uri_mismatch	Wrong credential type in Google Cloud	Delete credential, recreate as Desktop app type
ERR_MODULE_NOT_FOUND @tailwindcss/vite	Tailwind not installed as dev dep	npm install -D tailwindcss @tailwindcss/vite